

GSOI 2026 暑期常规赛

第四试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 灰草迷境 (spore)

T2 银淞止境 (rime)

T3 无终奇语 (fable)

T4 黑流树海 (tide)

吐槽时间

Contents

T1 灰蕈迷境 (spore)

T2 银淞止境 (rime)

T3 无终奇语 (fable)

T4 黑流树海 (tide)

吐槽时间

数据点 1 ~ 3 15Pts

- ▶ 这个时候 $n \leq 9$ ，直接枚举破除幻境的排列。对于一个固定顺序，从前向后模拟，若当前清明值不足，就将初始清明值补到恰好能够进入当前幻境。

数据点 1 ~ 3 15Pts

- ▶ 这个时候 $n \leq 9$ ，直接枚举破除幻境的排列。对于一个固定顺序，从前向后模拟，若当前清明值不足，就将初始清明值补到恰好能够进入当前幻境。
- ▶ 枚举排列的时间复杂度为 $\mathcal{O}(n!n)$ 。这一档也给出了接下来真正需要解决的问题：如何比较两段幻境的先后顺序。

数据点 4 ~ 6 15Pts

- 特殊性质 A 已经保证按照输入顺序破除幻境最优，因此不再需要枚举顺序，只需要完成上一档中对固定排列的模拟。

数据点 4 ~ 6 15Pts

- ▶ 特殊性质 A 已经保证按照输入顺序破除幻境最优，因此不再需要枚举顺序，只需要完成上一档中对固定排列的模拟。
- ▶ 依次考虑每座幻境，若当前清明值小于 a_i ，就将初始清明值增加两者之差，再令当前清明值变为 b_i 。总时间复杂度为 $\mathcal{O}(n)$ 。

数据点 7 ~ 11 25Pts

- 特殊性质 B 保证 $b_i \geq a_i$ ，破除幻境以后清明值不会减少。考虑两座满足 $a_i > a_j$ 的相邻幻境，交换以后先进入要求更低的第 j 座幻境，显然不会使后面的任意一步变得更难。

数据点 7 ~ 11 25Pts

- ▶ 特殊性质 B 保证 $b_i \geq a_i$ ，破除幻境以后清明值不会减少。考虑两座满足 $a_i > a_j$ 的相邻幻境，交换以后先进入要求更低的第 j 座幻境，显然不会使后面的任意一步变得更难。
- ▶ 因此按照 a_i 从小到大排序以后模拟即可。这里第一次利用交换相邻逆序对确定了顺序，而这个交换方法也会继续用于正解。

数据点 12 ~ 16 25Pts

- 特殊性质 C 保证 $b_i < a_i$ ，从正向看，每次破除幻境都会损失清明值。反过来考虑最终状态，若相邻两座幻境满足 $b_i < b_j$ ，那么将第 j 座幻境放在前面，能够为更早的过程留下更多清明值。

数据点 12 ~ 16 25Pts

- ▶ 特殊性质 C 保证 $b_i < a_i$ ，从正向看，每次破除幻境都会损失清明值。反过来考虑最终状态，若相邻两座幻境满足 $b_i < b_j$ ，那么将第 j 座幻境放在前面，能够为更早的过程留下更多清明值。
- ▶ 因此这一部分按照 b_i 从大到小排序。与上一档相同，排序以后只需要做一次线性模拟。

数据点 17 ~ 20 正解

- ▶ 回顾前两档特殊性质，将所有幻境分成 $b_i \geq a_i$ 与 $b_i < a_i$ 两类。第一类按照 a_i 升序排列，第二类按照 b_i 降序排列；同时，第一类破除以后不会减少清明值，因此必然放在第二类以前。

数据点 17 ~ 20 正解

- ▶ 回顾前两档特殊性质，将所有幻境分成 $b_i \geq a_i$ 与 $b_i < a_i$ 两类。第一类按照 a_i 升序排列，第二类按照 b_i 降序排列；同时，第一类破除以后不会减少清明值，因此必然放在第二类以前。
- ▶ 任意一个不满足上述顺序的相邻对都可以交换，并且交换以后所需的初始清明值不会增加。不断消去逆序对便能够得到上述排列，因此排序以后模拟即为最优解。总时间复杂度为 $\mathcal{O}(n \log n)$ ，答案与模拟过程均需要使用 long long。

正解 100Pts

```
1 bool cmp(point x,point y) {  
2     if (x.a<=x.b && y.a>y.b) return true;  
3     if (x.a>x.b && y.a<=y.b) return false;  
4     else if (x.a<=x.b && y.a<=y.b) return x.a<y.a;  
5     else return x.b>y.b;  
6 }
```

```
1 sort(P+1,P+n+1,cmp);  
2 for (int i=1;i<=n;i++) {  
3     if (P[i].a>Now){Ans+=(1ll*P[i].a-Now);Now=1ll*P[i].a;}  
4     Now-=1ll*P[i].a;Now+=1ll*P[i].b;  
5 }
```


数据点 1 ~ 4 16Pts

- 这个时候 $n \leq 10, m \leq 20$ ，枚举选择的 $n - 1$ 条道路，用并查集判断是否形成生成树，再检查所有关键观测站的度数以及与核心科考站直接相连的数量。

数据点 1 ~ 4 16Pts

- ▶ 这个时候 $n \leq 10, m \leq 20$ ，枚举选择的 $n - 1$ 条道路，用并查集判断是否形成生成树，再检查所有关键观测站的度数以及与核心科考站直接相连的数量。
- ▶ 直接枚举的时间复杂度为 $\mathcal{O}(2^m(n + m))$ 。观察合法方案中关键观测站只能作为叶子，这正是正解拆开处理这些点的基础。

数据点 5 ~ 8 16Pts

- 特殊性质 A 保证只有一个关键观测站。删去这个点以后，剩余科考站之间仍然必须连通，因此先对所有非关键观测站求一棵最小生成树。

数据点 5 ~ 8 16Pts

- ▶ 特殊性质 A 保证只有一个关键观测站。删去这个点以后，剩余科考站之间仍然必须连通，因此先对所有非关键观测站求一棵最小生成树。
- ▶ 最后只需要为关键观测站选择一条道路。若 $c = 0$ ，只能选择连向其他非关键观测站的最小边；否则还可以与连向核心科考站的最小边取较小值。时间复杂度为 $\mathcal{O}(m \log m)$ 。

数据点 9 ~ 13 20Pts

- ▶ 特殊性质 B 保证 $c = 0$ ，所有关键观测站都不能直接连接核心科考站。仍然先删去所有关键观测站，对剩余部分求最小生成树。

数据点 9 ~ 13 20Pts

- ▶ 特殊性质 B 保证 $c = 0$ ，所有关键观测站都不能直接连接核心科考站。仍然先删去所有关键观测站，对剩余部分求最小生成树。
- ▶ 每个关键观测站只能作为叶子，并且其选择不会影响其他关键观测站，因此分别选择一条连向非核心、非关键观测站的最小边即可。这一档将上一档对单个关键点的处理推广到了所有关键点。

数据点 14 ~ 18 20Pts

- ▶ 特殊性质 C 保证删去关键观测站以后，剩余道路已经是一棵树，所以生成树的骨架已经确定，直接累加这些道路的权值即可。

数据点 14 ~ 18 20Pts

- ▶ 特殊性质 C 保证删去关键观测站以后，剩余道路已经是一棵树，所以生成树的骨架已经确定，直接累加这些道路的权值即可。
- ▶ 对每个关键观测站，分别记连向核心的最小代价与连向其他非关键点的最小代价。先选择后者；若改为连接核心能够减少代价，就得到一个收益。将所有正收益从大到小排序，在额度内依次选择即可。

数据点 19 ~ 25 正解

- 结合以上，删去关键观测站以后，使用 Kruskal 求出剩余图的最小生成树。由于关键观测站的度数必须为 1，它们不可能参与连接剩余图中的两个连通块，所以这棵最小生成树一定是最优骨架。

数据点 19 ~ 25 正解

- ▶ 结合以上，删去关键观测站以后，使用 Kruskal 求出剩余图的最小生成树。由于关键观测站的度数必须为 1，它们不可能参与连接剩余图中的两个连通块，所以这棵最小生成树一定是最优骨架。
- ▶ 接下来沿用上一档对额度的处理。若某个关键观测站不存在连向其他非关键点的道路，它必须连接核心并消耗一个额度；其余点先选择非核心道路，再按照收益从大到小使用剩余额度。总时间复杂度为 $\mathcal{O}(m \log m + k \log k)$ ，答案需要使用 long long。

正解 100Pts

```
1  int Used=0;
2  for (int i=1;i<=n;i++) {
3      if (!Key[i]) continue;
4      if (Min[i][0]==0x3f3f3f3f) {
5          Ans+=1ll*Min[i][1];
6          Used++;
7      }
8      else {
9          Ans+=1ll*Min[i][0];
10         if (Min[i][1]<Min[i][0]) {
11             Diff[++cnt]=Min[i][1]-Min[i][0];
12         }
13     }
14 }
15 sort(Diff+1,Diff+cnt+1);
16 for (int i=1;i<=cnt&& i<=c-Used;i++) {
17     Ans+=Diff[i];
18 }
```

Contents

T1 灰草迷境 (spore)

T2 银淞止境 (rime)

T3 无终奇语 (fable)

T4 黑流树海 (tide)

吐槽时间

数据点 1 ~ 3 15Pts

- 枚举最终攻击区间 $[l, r]$ ，再枚举技能覆盖的子段，直接计算没有被技能消灭的敌人总生命值。若这个值不超过 c ，当前攻击区间合法。

数据点 1 ~ 3 15Pts

- ▶ 枚举最终攻击区间 $[l, r]$ ，再枚举技能覆盖的子段，直接计算没有被技能消灭的敌人总生命值。若这个值不超过 c ，当前攻击区间合法。
- ▶ 由于 $n \leq 20$ ，直接枚举即可。观察一个固定的攻击区间，当区间长度至少为 k 时，技能显然应该覆盖其中生命值之和最大的、长度恰好为 k 的子段。

数据点 4 ~ 7 20Pts

- ▶ 预处理前缀和以后，任意子段的生命值之和都可以在 $\mathcal{O}(1)$ 的时间内求出。固定左端点并向右枚举右端点，同时记录当前区间内最大的长度为 k 的子段和。

数据点 4 ~ 7 20Pts

- ▶ 预处理前缀和以后，任意子段的生命值之和都可以在 $\mathcal{O}(1)$ 的时间内求出。固定左端点并向右枚举右端点，同时记录当前区间内最大的长度为 k 的子段和。
- ▶ 每次右端点增加，只会多出一个新的候选技能区间，因此维护最大值以后，判断当前攻击区间只需要 $\mathcal{O}(1)$ 。枚举所有左右端点的时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 8 ~ 11 20Pts

- ▶ 特殊性质 A 保证 $k = 1$ ，技能只会消灭生命值最大的一个敌人。固定右端点时，若当前区间不合法，那么继续向左扩展只会使区间总生命值增加，而最大值的增量不会超过新加入的生命值，因此更大的区间仍然不合法。

数据点 8 ~ 11 20Pts

- ▶ 特殊性质 A 保证 $k = 1$ ，技能只会消灭生命值最大的一个敌人。固定右端点时，若当前区间不合法，那么继续向左扩展只会使区间总生命值增加，而最大值的增量不会超过新加入的生命值，因此更大的区间仍然不合法。
- ▶ 于是可以使用双指针维护最靠左的合法位置，并用单调队列维护区间最大值。左右端点都只会单调向右移动，时间复杂度为 $\mathcal{O}(n)$ 。

数据点 12 ~ 15 20Pts

- 特殊性质 B 保证生命值单调不降，因此一个长度至少为 k 的攻击区间中，生命值之和最大的长度为 k 的子段一定是最后 k 个位置。

数据点 12 ~ 15 20Pts

- ▶ 特殊性质 B 保证生命值单调不降，因此一个长度至少为 k 的攻击区间中，生命值之和最大的长度为 k 的子段一定是最后 k 个位置。
- ▶ 这个时候技能区间已经可以直接由右端点确定，仍然使用双指针维护最靠左的合法位置即可。与上一档相比，需要维护的最大值被单调性直接消去了。

数据点 16 ~ 20 正解

- ▶ 回顾第二档部分分，设当前区间总生命值为 S ，其中最大的长度为 k 的子段和为 M ，合法条件就是 $S - M \leq c$ 。固定右端点时，合法左端点仍然形成一个后缀，因此继续使用双指针。

数据点 16 ~ 20 正解

- ▶ 回顾第二档部分分，设当前区间总生命值为 S ，其中最大的长度为 k 的子段和为 M ，合法条件就是 $S - M \leq c$ 。固定右端点时，合法左端点仍然形成一个后缀，因此继续使用双指针。
- ▶ 每次右端点增加，只会加入一个新的长度为 k 的子段；左端点增加时，只会删除起点已经落在区间外的子段。使用单调队列维护所有候选子段和，并保持队首最大，每个子段至多进入、离开队列各一次，总时间复杂度为 $\mathcal{O}(n)$ 。

正解 100Pts

```
1      while (p<n&&Pre[p+1]-Pre[i-1]-Max<=c) {  
2          p++;  
3          while (Q.Size()&&Calc(Q.Back())<=Calc(p)) Q.PopB();  
4          Q.Push(p);  
5          if(p<n) Max=max(Calc(p+1),Calc(Q.Front()));  
6      }  
7      Ans=max(p-i+1,Ans);  
8  }
```

- 注意区间长度不超过 k 时，可以直接用技能斩杀整个区间，此时额外伤害为 0。前缀和与 c 均需要使用 long long。

Contents

T1 灰草迷境 (spore)

T2 银淞止境 (rime)

T3 无终奇语 (fable)

T4 黑流树海 (tide)

吐槽时间

数据点 1 ~ 5 20Pts

- 固定起点以后，所有必须经过的边恰好组成连接起点与全部调查员的最小子树。若最终还要返回起点，每条边都必须经过两次；而在接到最后一名调查员后可以直接结束，因此可以选择一条路径只走一次。

数据点 1 ~ 5 20Pts

- ▶ 固定起点以后，所有必须经过的边恰好组成连接起点与全部调查员的最小子树。若最终还要返回起点，每条边都必须经过两次；而在接到最后一名调查员后可以直接结束，因此可以选择一条路径只走一次。
- ▶ 显然应该选择起点到调查员的最长路径。对每个起点单独 DFS，求出最小子树边权和与最远调查员距离即可，单个起点 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 6 ~ 9 16Pts

- 特殊性质 A 保证只有一名调查员。无论从哪个区域出发，最优路线都只需要沿着简单路径到达这名调查员，因此答案就是两点之间的距离。

数据点 6 ~ 9 16Pts

- ▶ 特殊性质 A 保证只有一名调查员。无论从哪个区域出发，最优路线都只需要沿着简单路径到达这名调查员，因此答案就是两点之间的距离。
- ▶ 以调查员所在区域为根进行一次 DFS，求出其到所有区域的距离即可。总时间复杂度为 $\mathcal{O}(n)$ 。

数据点 10 ~ 14 20Pts

- 特殊性质 B 保证每个区域都有调查员，所以最小子树就是整棵树。设所有边权和为 S ，从区域 u 出发的答案为 $2S$ 减去 u 到最远区域的距离。

数据点 10 ~ 14 20Pts

- ▶ 特殊性质 B 保证每个区域都有调查员，所以最小子树就是整棵树。设所有边权和为 S ，从区域 u 出发的答案为 $2S$ 减去 u 到最远区域的距离。
- ▶ 树上任意一点距离最远的点一定可以在一组直径端点中取到。先用两次 DFS 找到直径端点，再分别求出两个端点到所有区域的距离，即可在线性时间内得到所有答案。

数据点 15 ~ 19 20Pts

- ▶ 特殊性质 C 保证整棵树是一条链。找到最左与最右的调查员以后，对于任意起点，必须经过的部分就是包含起点与这两个位置的最小区间。

数据点 15 ~ 19 20Pts

- ▶ 特殊性质 C 保证整棵树是一条链。找到最左与最右的调查员以后，对于任意起点，必须经过的部分就是包含起点与这两个位置的最小区间。
- ▶ 预处理链上的前缀距离，便可以求出这个区间的总长度以及起点到两个端点的距离。答案仍然是区间长度的两倍减去较大的端点距离，总时间复杂度为 $\mathcal{O}(n)$ 。

数据点 20 ~ 25 正解

- 回顾第一档部分分，设 F_u 表示从 u 出发，访问全部调查员后再回到 u 的最小代价。若从 u 到最远调查员的距离为 D_u ，那么最后这一段可以只走一次，因此

$$\text{Ans}(u) = F_u - D_u.$$

数据点 20 ~ 25 正解

- 回顾第一档部分分，设 F_u 表示从 u 出发，访问全部调查员后再回到 u 的最小代价。若从 u 到最远调查员的距离为 D_u ，那么最后这一段可以只走一次，因此

$$\text{Ans}(u) = F_u - D_u.$$

- 逐个起点 DFS 会产生大量重复计算。考虑先以 1 号区域为根求出子树内的信息，再通过第二次 DFS 完成换根。

第一次 DFS：子树 DP

- 设 $\text{Size}(u)$ 表示 u 的子树内调查员数量， f_u 表示从 u 出发，访问子树内全部调查员后回到 u 的最小代价。记调查员集合为 K ，则

$$\text{Size}(u) = [u \in K] + \sum_{v \in \text{Son}(u)} \text{Size}(v).$$

第一次 DFS: 子树 DP

- 设 $\text{Size}(u)$ 表示 u 的子树内调查员数量, f_u 表示从 u 出发, 访问子树内全部调查员后回到 u 的最小代价。记调查员集合为 K , 则

$$\text{Size}(u) = [u \in K] + \sum_{v \in \text{Son}(u)} \text{Size}(v).$$

- 只有包含调查员的儿子方向需要往返经过, 所以

$$f_u = \sum_{\substack{v \in \text{Son}(u) \\ \text{Size}(v) > 0}} (f_v + 2w(u, v)).$$

叶子处的空和为 0, 第一次 DFS 按照后序顺序转移。

第一次 DFS：最远距离

- 设 $d_{u,0}$ 、 $d_{u,1}$ 分别表示 u 到子树内调查员的最大、次大距离。先记所有可能从 u 出发的距离为

$$\mathcal{D}_u = \{0 \mid u \in K\} \\ \cup \{d_{v,0} + w(u, v) \mid v \in \text{Son}(u), \text{Size}(v) > 0\}.$$

于是

$$d_{u,0} = \max \mathcal{D}_u, \quad d_{u,1} = \text{secondmax } \mathcal{D}_u.$$

第一次 DFS：最远距离

- 设 $d_{u,0}$ 、 $d_{u,1}$ 分别表示 u 到子树内调查员的最大、次大距离。先记所有可能从 u 出发的距离为

$$\mathcal{D}_u = \{0 \mid u \in K\} \\ \cup \{d_{v,0} + w(u, v) \mid v \in \text{Son}(u), \text{Size}(v) > 0\}.$$

于是

$$d_{u,0} = \max \mathcal{D}_u, \quad d_{u,1} = \text{secondmax } \mathcal{D}_u.$$

- 若不存在次大值，则在数学上将其视为 $-\infty$ ；代码中使用 0 作为初值，并结合当前点是否为调查员判断这个 0 是否有效。

换根时维护代价

- 第二次 DFS 中，先令 $F_1 = f_1$ 。设当前点为 u ，其儿子为 v ，调查员总数为 k 。从 u 换根到 v 时有

$$F_v = \begin{cases} F_u + 2w(u, v), & \text{Size}(v) = 0, \\ f_v, & \text{Size}(v) = k, \\ F_u, & 0 < \text{Size}(v) < k. \end{cases}$$

换根时维护代价

- 第二次 DFS 中，先令 $F_1 = f_1$ 。设当前点为 u ，其儿子为 v ，调查员总数为 k 。从 u 换根到 v 时有

$$F_v = \begin{cases} F_u + 2w(u, v), & \text{Size}(v) = 0, \\ f_v, & \text{Size}(v) = k, \\ F_u, & 0 < \text{Size}(v) < k. \end{cases}$$

- 三种情况分别表示父边需要新增为往返边、全部调查员都在 v 的子树内，以及连接全部调查员的最小子树没有变化。

换根时维护最远距离

- 设 g_u 表示从 u 经过父亲方向到调查员的最大距离，根节点满足 $g_1 = -\infty$ 。排除儿子 v 以后，令

$$h_{u,v} = \max \begin{cases} g_u, \\ 0, \\ d_{x,0} + w(u,x), \end{cases} \quad \begin{matrix} u \in K, \\ x \in \text{Son}(u) \setminus \{v\}, \\ \text{Size}(x) > 0. \end{matrix}$$

则 $g_v = h_{u,v} + w(u,v)$ ；若没有合法候选，则令 $g_v = -\infty$ 。

换根时维护最远距离

- 设 g_u 表示从 u 经过父亲方向到调查员的最大距离，根节点满足 $g_1 = -\infty$ 。排除儿子 v 以后，令

$$h_{u,v} = \max \begin{cases} g_u, \\ 0, \\ d_{x,0} + w(u,x), \end{cases} \quad \begin{matrix} u \in K, \\ x \in \text{Son}(u) \setminus \{v\}, \\ \text{Size}(x) > 0. \end{matrix}$$

则 $g_v = h_{u,v} + w(u,v)$ ；若没有合法候选，则令 $g_v = -\infty$ 。

- 实现时，若 u 的最大值来自 v ，便使用 $d_{u,1}$ ；否则使用 $d_{u,0}$ 。于是 $D_u = \max(d_{u,0}, g_u)$ 。

换根的关键实现

```
1 if(Size[Now]==0) F[Now]=F[From]+2*LastLen;
2 else if(k-Size[Now]==0) F[Now]=Dp[Now];
3 else F[Now]=F[From];
4 long long NowD;
5 if(Size[Now]&&Dis[From][0]==LastLen+Dis[Now][0]){
6     NowD=Dis[From][1]+LastLen;
7     if(Dis[From][1]==0&&!Key[From]) NowD=-1;
8 }
9 else NowD=Dis[From][0]+LastLen;
```

正解 100Pts

- 第二次 DFS 按照从父亲到儿子的顺序转移。得到 F_u 与父亲方向的最远距离以后，最终答案为

$$\text{Ans}(u) = F_u - \max(d_{u,0}, g_u).$$

正解 100Pts

- 第二次 DFS 按照从父亲到儿子的顺序转移。得到 F_u 与父亲方向的最远距离以后，最终答案为

$$\text{Ans}(u) = F_u - \max(d_{u,0}, g_u).$$

- 两次 DFS 都只会经过每条边常数次，时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。边权和可能达到 10^{14} ，所有距离与答案必须使用 long long。

Contents

T1 灰蕈迷境 (spore)

T2 银淞止境 (rime)

T3 无终奇语 (fable)

T4 黑流树海 (tide)

吐槽时间